VibeCoding 项目文档管理规范

> Agent 依据本文档维护项目全生命周期文档。
> 写入项目根目录 `CLAUDE.md` 或 `.project/CONVENTIONS.md` 即可生效。

---

1. 核心铁律

> **PRD 是一切代码改动的前提，包括 Bug 修复。** Agent 必须在动手写代码前检查 PRD 是否存在，不存在则先创建。

#	铁律	说明
1	**无 PRD 不工作**	没有 PRD 文档，Agent 必须拒绝编码（包括 Bug 修复），先让用户确认需求并写 PRD
2	**修改必留痕**	任何文档改动，必须在该文档的变更表追加一行
3	**API 必须最新可用**	`.project/api/` 下的文档任何时候都必须与代码一致
4	**合并必解冲突**	合并代码时，必须同步解决文档冲突，不允许文档落后于代码
5	**先 PRD 后代码**	Agent 在写任何代码之前必须先确认 PRD 已存在，不存在则先创建并让用户确认

---

2. 文件夹结构

```
<project-root>/
├── .project/                                   # 项目文档管理（.gitignore 可选）
|   ├── prd/                                  # PRD（一切工作的入口）
|   |   ├── project/                        # 项目级 PRD
|   |   |   └── <项目描述>-vN.md
|   |   ├── module/                         # 模块级 PRD
|   |   |   └── <module-name>/
|   |   |       └── <模块描述>-vN.md
|   |   ├── feature/                        # 功能级 PRD
|   |   |   └── <module-name>/
|   |   |       └── <feature-name>-vN.md
|   |   └── bugfix/                         # Bug 修复级 PRD
|   |       └── <module-name>/
|   |           └── bugfix-<简述>-vN.md
|   ├── architecture/                         # 总体架构（仅项目级 PRD 触发）
|   |   └── <架构名称>-vN.md
|   ├── modules/                              # 模块（自包含）
|   |   └── <module-name>/
|   |       ├── <模块描述>-vN.md            # 模块设计
|   |       └── features/                     # 该模块下的所有功能
|   |           └── <feature-name>/
|   |               ├── design.md             # 功能设计
|   |               ├── changelog.md          # 该功能的变更记录
```

```
|   |                 └── bugfix.md          # 该功能的 bug 记录
|   ├── api/                                 # API（全局，必须实时同步）
|   |   ├── _overview.md                     # 通用约定
|   |   └── <resource>.md                    # 每个资源一个文件
|   ├── db/
|   |   └── schema.md
|   └── changelog/
|       ├── CHANGELOG.md                     # 全局变更日志
|       └── adr/
|           └── NNNN-<标题>.md
├── CLAUDE.md
└── src/
```

设计原则：

- **模块自包含** — 每个模块是一个独立单元，它下面的功能设计、变更记录、bugfix 都在模块文件夹内，不散落到外面
- **PRD 按层级分目录** — 四级目录（project / module / feature / bugfix），通过目录位置体现归属关系，文件名保留 `-vN` 版本后缀

---

3. PRD 分级

级别	目录	适用场景	触发链路
项目级	`prd/project/`	项目立项、大方向变更、新增业务线	PRD → 架构 → 模块 → 功能 → API
模块级	`prd/module/<module-name>/`	新增模块、模块拆分/合并、模块职责重大调整	PRD → 模块 → 功能 → API（跳过架构）
功能级	`prd/feature/<module-name>/`	模块内功能增删改、小需求迭代	PRD → 功能设计 → API（跳过架构和模块设计）
Bug 修复级	`prd/bugfix/<module-name>/`	线上 bug、用户反馈、测试发现的缺陷	PRD → bugfix → changelog（跳过架构和功能设计）

判断：

- 修 bug、补漏洞、修复用户反馈 → **Bug 修复级**
- 只改某个模块内的功能 → **功能级**
- 新增模块或模块职责调整 → **模块级**
- 涉及跨模块重组、技术栈切换、整体架构变更 → **项目级**
- 拿不准 → 问用户

---

4. 文档模板

4.1 PRD

项目级：`prd/project/<项目描述>-vN.md`

```
# PRD [项目级] — <标题>

> 版本：<标题>-vN
> 日期：YYYY-MM-DD
> 指令人：@用户名
> 上一版本：<标题>-v<N-1>（首次填「无」）

## 背景
要解决什么问题、目标用户

## 需求列表
| # | 需求 | 指令人 | 状态 | 功能设计 |
|---|------|--------|------|---------|
| 1 | ... | @xxx | 开发中 | modules/auth/features/user-login/design.md |

## 驱动
→ architecture/<架构名称>-vN.md

## 变更
| 日期 | 内容 | 指令人 |
|------|------|--------|
```

模块级：`prd/module/<module-name>/<模块描述>-vN.md`

```
# PRD [模块级] — <标题>

> 版本：<模块描述>-vN
> 日期：YYYY-MM-DD
> 指令人：@用户名
> 归属模块：modules/<module-name>

## 背景
为什么需要这个模块、解决什么问题

## 职责
一句话描述模块的职责边界

## 功能规划
| # | 功能 | 优先级 | 功能设计 |
|---|------|--------|---------|
| 1 | ... | P0 | modules/<module-name>/features/<feature>/design.md |

## 模块设计
→ modules/<module-name>/<模块描述>-vN.md
```

```
## 变更
| 日期 | 内容 | 指令人 |
|------|------|--------|
```

功能级：`prd/feature/<module-name>/<feature-name>-vN.md`

```
# PRD [功能级] — <标题>

> 版本：<feature-name>-vN
> 日期：YYYY-MM-DD
> 指令人：@用户名
> 归属模块：modules/<module-name>

## 需求
做什么、为什么做、预期效果

## 功能设计
→ modules/<module-name>/features/<feature-name>/design.md

## 变更
| 日期 | 内容 | 指令人 |
|------|------|--------|
```

Bug 修复级：`prd/bugfix/<module-name>/bugfix-<简述>-vN.md`

```
# PRD [Bug 修复级] — <标题>

> 版本：bugfix-<简述>-vN
> 日期：YYYY-MM-DD
> 指令人：@用户名
> 反馈人：@用户名
> 归属模块：modules/<module-name>
> 严重程度：Critical / Major / Minor

## 问题描述
- **期望行为**：应该怎样
- **实际行为**：实际怎样
- **复现步骤**：1. ... 2. ... 3. ...

## 根因分析
定位到哪个文件/函数/逻辑出了问题

## 修复方案
改什么、怎么改、为什么这样改

## 影响范围
- [ ] 涉及 API 变更 → api/<resource>.md
- [ ] 涉及数据结构变更 → db/schema.md
- [ ] 涉及功能设计变更 → modules/<name>/features/<feature>/design.md

## 验证方式
如何确认修复生效
```

```
## 变更
| 日期 | 内容 | 指令人 |
|------|------|--------|
```

4.2 架构：`architecture/<架构名称>-vN.md`

```
# 架构 – <标题>-vN

> 版本：<标题>-vN
> 日期：YYYY-MM-DD
> 指令人：@用户名
> 来源 PRD：prd/project/<项目描述>-vN

## 全景图
\```
[架构图]
\```

## 技术栈
| 层级 | 选型 | 版本 | 理由 |
|------|------|------|------|

## 模块划分
| 模块 | 职责 | 设计文档 |
|------|------|---------|

## 通信方式
内部调用、外部依赖

## 全局约束
鉴权、错误码、限流等

## 变更
| 日期 | 内容 | 指令人 |
|------|------|--------|
```

> 只有项目级 PRD 才触发架构更新。

4.3 模块：`modules/<name>/<模块描述>-vN.md`

```
# 模块 – <名称> <描述>-vN

> 版本：<描述>-vN
> 日期：YYYY-MM-DD
> 指令人：@用户名
> 来源架构：architecture/<架构描述>-vN

## 职责
```

```
一句话

## 架构
\```
[模块架构图]
\```

## 对外接口
| 方法 | 输入 | 输出 | 说明 |
|------|------|------|------|

## 内部组件
| 组件 | 职责 | 文件 |
|------|------|------|

## 功能列表
| 功能 | 状态 | 文档 |
|------|------|------|
| 用户登录 | 已完成 | features/user-login/design.md |
| 手机号登录 | 开发中 | features/phone-login/design.md |

## 绑定的 API
- api/auth.md

## 变更
| 日期 | 内容 | 指令人 |
|------|------|--------|
```

> 模块升版本时，逐个检查「功能列表」中的功能是否受影响，受影响的更新其 design.md 头部。

4.4 功能设计：`modules/<name>/features/<feature>/design.md`

```
# 功能 — <名称>

> 日期：YYYY-MM-DD
> 指令人：@用户名
> 来源 PRD：prd/feature/<module-name>/<feature-name>-vN
> 所属模块：modules/<name>/<当前版本>

## 描述
做什么、预期效果

## 依赖

## 实现逻辑
### 正常流程
1. ...
### 异常流程
1. ...

## 涉及 API
```

```
→ api/<resource>.md

## 涉及数据
→ db/schema.md

## 变更
→ changelog.md
```

4.5 功能 Changelog：`modules/<name>/features/<feature>/changelog.md`

```
# 变更记录 – <功能名称>

| 日期 | 内容 | 指令人 | 触发来源 |
|------|------|--------|---------|
| YYYY-MM-DD | 初始设计 | @xxx | 新功能 |
| YYYY-MM-DD | 追加手机号字段 | @xxx | 需求变更 |
| YYYY-MM-DD | 修复空指针异常 | @xxx | BUG-002 |
```

> 每次改动必追加一行，不留空的变更表。
> 触发来源：新功能 / 需求变更 / 模块升级 / BUG修复 / 代码审计 / 重构。
>
> **跨模块变更规则**：当一个 PRD 影响多个功能时，必须为每个受影响功能的 `changelog.md` 追加条目，不能只更新全局 CHANGELOG。

4.6 功能 Bugfix：`modules/<name>/features/<feature>/bugfix.md`

```
# Bug 记录 – <功能名称>

## BUG-001: <标题>
- **日期**: YYYY-MM-DD
- **严重程度**: Critical / Major / Minor
- **发现人**: @xxx
- **问题**: 期望 vs 实际
- **根因**: ...
- **修复**: 改了什么
- **影响文档**:
  - [x] design.md
  - [x] api/<resource>.md
  - [ ] db/schema.md

---

## BUG-002: <标题>
（同上）

---
```

```
## 统计
| 严重程度 | 数量 |
|---------|------|
| Critical | 0 |
| Major | 0 |
| Minor | 0 |
```

4.7 API：`api/<resource>.md`

```
# API – <资源名称>

> 最后同步：YYYY-MM-DD
> **本文档必须与代码保持一致，任何接口变更必须立即更新**

## 认证
...

## POST /api/v1/<resource>
**说明**：创建 xxx

请求：
\```json
{ "field": "string" }
\```

响应 201：
\```json
{ "id": "string" }
\```

错误：
| 状态码 | code | 说明 |
|--------|------|------|

---

## GET /api/v1/<resource>/:id
（同上）

---

## 变更
| 日期 | 内容 | 指令人 | 来源功能 |
|------|------|--------|---------|
| YYYY-MM-DD | 新增 POST 接口 | @xxx | modules/auth/features/user-login |
| YYYY-MM-DD | 修改响应字段 | @xxx | modules/auth/features/phone-login |
```

> **铁律**：API 文档任何时候都必须与实际代码一致。改了接口不改文档 = 没做完。
> 每次接口变更，必须立即在此文件变更表追加一行，标注来源功能。

4.8 DB：`db/schema.md`

```
# 数据模型

> 最后同步：YYYY-MM-DD

## ER 图
\```
[Mermaid erDiagram]
\```

## <table_name>
> 来源：modules/<name>/<当前版本>

| 字段 | 类型 | 约束 | 说明 |
|------|------|------|------|

索引：

## 变更
| 日期 | 内容 | 指令人 | 来源 |
|------|------|--------|------|
```

4.9 全局 Changelog：`changelog/CHANGELOG.md`

```
# Changelog

## [Unreleased]
### Added
- <功能> — modules/xxx/features/yyy/design.md

### Refactored
- <重构内容> — `src/path/to/file.ts`

### Changed
- <变更> — modules/xxx/<模块版本>

### Fixed
- BUG-001: <描述> — modules/xxx/features/yyy/bugfix.md

### Breaking
- **BREAKING** <描述> — changelog/adr/NNNN-<标题>.md

---

## [1.0.0] - 2025-01-01
### Added
- 初始版本

[Unreleased]: ...
```

```
[1.0.0]: ...
```

4.10 ADR：`changelog/adr/NNNN-<标题>.md`

```
# ADR-0001 – <标题>

> 日期：YYYY-MM-DD
> 指令人：@用户名

## 状态
提议 / 已采纳 / 已废弃

## 背景
## 决策

## 备选
| 方案 | 优点 | 缺点 |
|------|------|------|

## 影响文档
- [ ] architecture/<版本>
- [ ] modules/<name>/<版本>
- [ ] modules/<name>/features/<feature>/design.md
- [ ] api/<resource>.md
```

---

5. Agent 维护协议

5.1 铁律执行

```
用户提出需求
    |
    ├─ 没有对应 PRD？→ 拒绝工作，提示用户先确认需求，写 PRD
    |
    └─ 有 PRD？→ 继续
        |
        ├─ 改动了代码？→ 必须同步更新文档，并在变更表留痕
        |
        ├─ 改了 API？→ api/ 文件必须立即更新，否则任务不算完成
        |
        └─ 合并代码？→ 必须检查并解决文档冲突
```

5.2 新项目初始化

```
1. 创建 .project/ 目录结构
2. 问用户：「需求由谁发起？」→ 写入指令人
3. 写 prd/project/<项目描述>-v1.md → 用户确认
4. 写 architecture/<架构描述>-v1.md → 确认指令人 → 用户确认
5. 创建 modules/<name>/，每个模块：
   - 写 <模块描述>-v1.md
   - 创建 features/ 子目录
```

5.3 新模块开发

```
1. 检查是否有对应 PRD → 没有则拒绝，先写 PRD
2. 问用户「谁要求的？」→ 写入指令人
3. 写 prd/module/<module-name>/<模块描述>-v1.md → 用户确认
4. 写 modules/<name>/<模块描述>-v1.md
5. 创建 features/ 子目录
```

5.4 新功能开发

```
1. 检查是否有对应 PRD → 没有则拒绝，先写 PRD
2. 问用户「谁要求的？」→ 写入指令人
3. 判断级别：

   功能级：
   - 写 prd/feature/<module-name>/<feature-name>-v1.md
   - 写 modules/<name>/features/<feature>/design.md
   - 写 modules/<name>/features/<feature>/changelog.md（初始行）
   - 更新 modules/<name>/<模块版本>.md 的功能列表
   - 更新 api/<resource>.md（立即同步，变更表留痕）
   - 更新 db/schema.md
   - 全局 CHANGELOG 记录

   模块级：
   - 写 prd/module/<module-name>/<模块描述>-v1.md
   - 评估是否更新 modules/<name>/ 模块版本
   - 再走功能级流程

   项目级：
   - 写 prd/project/<项目描述>-vN.md
   - 评估是否更新 architecture/
   - 评估是否更新 modules/<name>/ 模块版本
   - 再走功能级流程
```

5.5 Bug 修复

```
0. **先写 PRD 再写代码** — 这是铁律，修 Bug 也必须先创建 PRD
1. 问用户「谁反馈的？」
2. 写 prd/bugfix/<module-name>/bugfix-<简述>-v1.md（使用 Bug 修复级 PRD 模板）
   - 填写：问题描述、期望/实际行为、复现步骤
3. 让用户确认 PRD
4. 定位根因，在 PRD 中记录分析结果和修复方案
5. 确定属于哪个模块的哪个功能
6. 开始编码修复
7. 在 modules/<name>/features/<feature>/bugfix.md 追加记录
8. 在 modules/<name>/features/<feature>/changelog.md 追加变更行
9. 如影响 API → 立即更新 api/<resource>.md 并留痕
10. 如影响 DB → 更新 db/schema.md
11. 全局 CHANGELOG → Fixed
```

5.6 合并代码

```
合并时 Agent 必须执行：
1. 检查代码变更涉及的模块和功能
2. 对比文档与代码：
   - API 签名一致？→ 不一致则以代码为准更新文档
   - DB 字段一致？→ 不一致则更新 schema.md
   - 功能逻辑一致？→ 不一致则更新 design.md
3. 解决文档冲突：
   - 两个分支改了同一份文档 → 合并内容，保留双方变更记录
   - 文档落后于代码 → 以代码为准补齐文档
4. 所有文档变更追加留痕行
5. 全局 CHANGELOG 记录合并内容
```

5.7 日常规则

规则	说明
无 PRD 不工作	没有需求文档，Agent 拒绝编码（包括 Bug 修复）
先 PRD 后代码	Agent 在写任何代码前必须先确认 PRD 存在，不存在则先创建
修改必留痕	任何文档改动都追加变更行
API 永远最新	`api/` 下文档任何时候都与代码一致
合并必解冲突	合并代码时同步解决文档差异
指令人必确认	创建/大改文档问用户「谁指挥的」
模块自包含	功能设计、changelog、bugfix 都在模块文件夹内
版本带描述	文件名 `<描述>-vN`，不纯 `v1/v2`
PRD 按层级分目录	`prd/project/`、`prd/module/<name>/`、`prd/feature/<name>/`、`prd/bugfix/<name>/`

规则	说明
跨模块逐功能更新 changelog	一个 PRD 影响多个功能时，必须为每个受影响功能的 `changelog.md` 追加条目

---

6. CLAUDE.md 集成

```
## 文档管理规范

> **PRD 优先于代码。** 这是本项目的最高优先级规则。任何代码改动（包括 Bug 修复）都必须先有 PRD。

所有文档位于 `.project/` 下。Agent **必须**遵守以下铁律：

1. **无 PRD 拒绝工作**：没有需求文档不写代码，Bug 修复也不例外
2. **修改必留痕**：任何文档改动追加变更行
3. **API 必须最新**：接口改了文档必须立即同步
4. **合并必解冲突**：合并代码时同步解决文档差异
5. **先 PRD 后代码**：Agent 必须在写任何代码之前先确认 PRD 已存在或创建 PRD

PRD 按层级分目录：
- `prd/project/` — 项目级（立项、架构变更）
- `prd/module/<name>/` — 模块级（新增/调整模块）
- `prd/feature/<name>/` — 功能级（模块内功能增删改）
- `prd/bugfix/<name>/` — Bug 修复级（通过目录位置体现归属）

其他规则：
- 每个模块自包含：功能设计、changelog、bugfix 在 `modules/<name>/features/<feature>/` 下
- 指令人必确认：创建/大改文档问用户
- 版本命名带描述：`<名称>-vN`
- Bug 记在对应功能的 `bugfix.md`，同时写 `prd/bugfix/<module>/bugfix-<简述>-vN.md`
- 全局变更记在 `.project/changelog/CHANGELOG.md`

详见：docs/vibecoding-doc-standard.md
```

---

7. 检查清单

- ☐ 有对应 PRD？（没有 → **停，先写 PRD**）
- ☐ 指令人已确认？
- ☐ 所有文档变更都留痕了？
- ☐ API 文档与代码一致？
- ☐ 模块内功能列表已更新？
- ☐ **所有受影响功能的 changelog.md 都追加了？**（跨模块任务容易遗漏，逐个检查）
- ☐ Bug 记到功能的 bugfix.md 了？
- ☐ DB schema 同步了？
- ☐ 全局 CHANGELOG 更新了？（Added / Refactored / Changed / Fixed / Breaking）

- [ ] 合并时文档冲突解决了？
- [ ] 有破坏性变更需要 ADR？

7.1 跨模块变更检查清单（附加）

当 PRD 影响多个模块/功能时，在标准检查清单基础上额外确认：

- [ ] 列出本次变更涉及的所有功能（features）清单
- [ ] 每个功能的 `changelog.md` 都追加了对应条目
- [ ] 全局 CHANGELOG 的 Refactored/Changed 段落按功能分组，标注涉及文件路径
- [ ] PRD 变更表记录完成状态

变更

日期	内容	指令人
2026-04-16	初始文档规范	@moonseeker1
2026-04-16	强化 PRD 优先级：增加第 5 条铁律「先 PRD 后代码」，Bug 修复流程增加步骤 0 强调先写 PRD，日常规则加粗前两条	@moonseeker1
2026-04-16	PRD 分四层目录（project/module/feature/bugfix），新增模块级 PRD 模板，Agent 协议更新路径引用	@moonseeker1
2026-04-16	新增跨模块变更规则：逐功能更新 changelog、全局 CHANGELOG 增加 Refactored 分类、触发来源扩展（代码审计/重构）、检查清单增加跨模块附加项	@moonseeker1